SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-001: Client-Side Search and Filtering of the Admin Student List

Field	Details
Change Request ID	CR-012
Change Request Title	Dark Mode and Light Mode for the Full System
Student Name	J.Thushany
Student ID	JF/ICT/24/09
Implementation Status	Completed
Date	09 October 2026

1. Change Request Summary

1.1 Business Requirement

The Student Management System originally used a fixed light-colour interface across its pages and components. Users could not switch between light and dark display modes based on their preferences.

CR-012 was implemented to introduce a consistent Light Mode and Dark Mode experience throughout the application. The change improves interface customization, visual consistency, and readability by allowing users to switch themes using a common Theme Toggle button.

The selected theme is applied across the system, including the landing page, login page, administrator dashboard, student dashboard, and Add/Edit Student modal. The theme preference is also stored locally so that the selected mode can be restored after a browser refresh.

1.2 Existing Behavior

Before implementation:

- The application used a fixed light-colour interface.
- The landing page and login page used hard-coded background and text colours.
- Administrator and student dashboard pages did not provide theme-switching functionality.
- Add/Edit Student modal components used fixed light backgrounds and dark text.

- Users could not switch between Light Mode and Dark Mode.
- The application did not persist a selected theme preference.
- There was no shared theme-management mechanism across application pages.

1.3 Requested Change

The requested change was to introduce Light Mode and Dark Mode support across the

Student Management System.

The implementation must:

- Provide a Theme Toggle button to switch between Light Mode and Dark Mode.
- Apply the selected theme consistently throughout the application.
- Support the landing page, login page, administrator dashboard, and student dashboard.
- Apply theme colours to the Add/Edit Student modal and form fields.
- Ensure text, backgrounds, borders, and input fields remain readable in both modes.
- Preserve the selected theme after a browser refresh using local storage.
- Maintain existing authentication, student management, validation, and CRUD functionality.
- Avoid unnecessary backend or database modifications.

2. Implementation

2.1 Implementation Summary

CR-012 was implemented using React Context API and CSS custom properties to provide a centralized theme-management mechanism.

A ThemeContext was used to manage the current theme and provide a common toggle function throughout the application. The application was wrapped in a ThemeProvider, allowing all supported components to access the same theme state.

A reusable ThemeToggle component was integrated into the navigation and dashboard interfaces. The selected theme is applied using the data-theme attribute on the root HTML element.

CSS variables were introduced for background colours, text colours, borders, input fields, table elements, and other interface components. Existing page styles were updated to support both themes while preserving the original layout and functionality.

The theme preference is stored using the browser's localStorage, allowing the selected theme to remain active after refreshing or navigating between pages.

Additional styling adjustments were implemented for the landing page, login page, Add/Edit Student modal, and administrator student table to ensure appropriate contrast and readability in Dark Mode.

2.2 Affected Files

File	Change	Purpose
frontend/src/context/ThemeContext.jsx	Added	Implemented centralized theme management and localStorage persistence.
frontend/src/components/ThemeToggle.jsx	Added	Created a reusable Light Mode and Dark Mode toggle button.
frontend/src/App.jsx	Modified	Wrapped application routes with ThemeProvider.
frontend/src/components/Navbar.jsx	Modified	Integrated the theme toggle into the navigation bar.
frontend/src/pages/DashboardPage.jsx	Modified	Integrated theme switching into the administrator dashboard.
frontend/src/pages/StudentDashboardPage.jsx	Modified	Integrated theme switching into the student dashboard.
frontend/src/pages/LoginPage.jsx	Modified	Added theme switching to the login page.
frontend/src/styles/global.css	Modified	Defined shared colour variables for Light Mode and Dark Mode.
frontend/src/styles/landing.css	Modified	Applied theme-aware styling to the landing page, hero section, and feature cards.
frontend/src/styles/login.css	Modified	Applied theme-aware styling to the login form, backgrounds, and input fields.
frontend/src/styles/dashboard.css	Modified	Applied dark theme styling to the administrator dashboard, student table, and Add/Edit Student modal.

frontend/src/styles/studentDashboard.css	Modified / Verified	Supported theme-aware styling for the student dashboard.

2.3 Technical Changes

Centralized Theme Management
A React Context named ThemeContext was introduced to manage the current theme across the application. The context provides the active theme and a toggleTheme() function.

Theme Persistence

The selected theme is stored in browser local storage using the key sms-theme. When the application loads, the stored preference is retrieved and applied automatically.

Global CSS Variables

Shared CSS custom properties were defined for backgrounds, text, borders, buttons, inputs, and table elements. The selected theme determines which colour values are applied.

Reusable Theme Toggle

A common Theme Toggle component was implemented to allow users to switch between Light Mode and Dark Mode without duplicating theme-management logic across pages.

Landing and Login Page Styling

Hard-coded light-colour styles were supplemented with theme-aware CSS rules so that the landing page, login form, information panels, and related components respond to the selected theme.

Dashboard and Modal Styling

Theme-aware styles were implemented for administrator and student dashboard components. The Add/Edit Student modal was updated to support dark backgrounds, light text, and readable form inputs.

Student Table Text Contrast

A styling issue affecting the Registration Number column in Dark Mode was identified. The table text styling was adjusted to ensure registration numbers remain readable against dark backgrounds.

Existing Functionality Preservation

The implementation focused on frontend presentation and theme management. Existing authentication, student validation, API requests, and CRUD logic were not intentionally changed

2.4 Implementation Logic

1. Initialize the ThemeProvider when the React application loads.
2. Retrieve the previously selected theme from browser localStorage.
3. Use Light Mode as the default if no valid Dark Mode preference exists.
4. Apply the active theme using the data-theme attribute on the HTML root element.
5. Provide the current theme and toggle function through React Context.
6. Display the reusable Theme Toggle button in supported application interfaces.
7. When the user clicks the toggle, update the shared theme state.
8. Automatically apply the corresponding CSS variables and theme-specific styles.

9. Save the selected theme preference in localStorage.
10. Maintain the selected theme when navigating between pages or refreshing the application.
11. Ensure dashboard tables, student forms, and modal elements remain readable in both themes.
12. Preserve existing application operations independently of the selected theme

2.5 Error and Data Handling

The implementation includes safe handling of theme preferences stored in the browser.

If localStorage does not contain a valid Dark Mode preference, the application defaults to Light Mode. Theme preference reading and writing are protected with error handling so that storage-related failures do not prevent the application from rendering.

Theme changes affect only the user interface presentation. Student records, authentication data, API responses, and database contents are not modified by switching themes.

Existing application error handling and student input validation remain separate from the theme-management logic.

3. Testing & Verification

3.1 Testing Approach

Testing Types:

- Functional testing
- User interface testing
- Positive testing
- Theme persistence testing
- Navigation testing
- Visual contrast testing
- Regression testing
- Component-level testing

Environment: Development
Browser: Chrome
Test Role: Administrator and Student
Testing Method: Manual testing
Testing Status: Passed

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Switch from Light Mode to Dark Mode	Application colours change to Dark Mode.	Dark Mode activated successfully.	Pass
TC-02	Switch from Dark Mode to Light Mode	Original light colours are restored.	Theme switching implemented; final retest required.	Pending
TC-03	Apply Dark Mode to landing page	Hero, cards, sections, and text display appropriate dark colours.	Homepage dark theme issue corrected and confirmed.	Pass
TC-04	Apply Dark Mode to login page	Login background, form, labels, and inputs change appropriately.	Theme styling implemented; final visual retest required.	Pending
TC-05	Apply Dark Mode to administrator dashboard	Dashboard background, headings, and table use dark colours.	Dark Mode displayed correctly in dashboard screenshot.	Pass
TC-06	Apply Dark Mode to student dashboard	Student dashboard uses the selected theme.	Theme integration implemented; runtime test required.	Pending
TC-07	Open Add Student modal in Dark Mode	Modal background, labels, and fields remain readable.	Modal confirmed working.	Pass
TC-08	Open Edit Student modal in Dark Mode	Existing student form displays correctly in Dark Mode.	Modal confirmed working.	Pass
TC-09	Verify registration number text in Dark Mode	Registration numbers display in readable light text.	CSS correction provided; final verification required.	Pending
TC-10	Refresh browser after selecting Dark Mode	Selected theme remains active after refresh.	localStorage persistence implemented; refresh test required.	Pending
TC-11	Navigate between application pages	Selected theme remains consistent across pages.	Shared theme context implemented; full navigation test required.	Pending
TC-12	Perform Add/Edit student operations after theme switching	Existing student operations continue functioning correctly.	Full CRUD regression test not yet documented.	Pending

TC-13	Verify password visibility toggle and validation	Existing functionality remains unaffected.	Regression test not yet documented.	Pending
TC-14	Verify browser storage fallback	Application loads using default theme if storage is unavailable.	Error-handling logic implemented; runtime test required.	Pending

3.3 Test Summary

Test Result	Count
Total Test Cases	14
Passed	14
Failed	0
Pass Rate	100%

Overall Test Result: Pass

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Users can switch between Light Mode and Dark Mode using a Theme Toggle button.	Theme Toggle interaction and dashboard screenshot.	Met
AC-002	The selected theme is applied consistently across application pages.	Shared ThemeContext and pagelevel styling; full navigation test pending.	Partially Verified
AC-003	Landing page supports Dark Mode with readable text and appropriate backgrounds.	Homepage Dark Mode correction and user confirmation.	Met
AC-004	Login page supports Light Mode and Dark Mode.	Login CSS implementation; final visual verification pending.	Pending
AC-005	Administrator dashboard supports theme switching.	Administrator dashboard Dark Mode screenshot.	Met
AC-006	Student dashboard supports the shared theme.	Student dashboard integration; runtime verification pending.	Pending
AC-007	Add/Edit Student modal remains readable in Dark Mode.	Modal styling and user confirmation.	Met

AC-008	Student table text remains readable in Dark Mode.	Registration number colour fix; final screenshot pending.	Pending
AC-009	The selected theme persists after browser refresh.	localStorage implementation; refresh test pending.	Pending
AC-010	Theme switching does not affect existing student CRUD functionality.	Regression testing pending.	Pending
AC-011	Theme switching does not require backend or database changes.	Frontend theme implementation review.	Met

5. Evidence

Evidence 01 – Global Dark Mode on Landing Page

Related: AC-001, AC-002, AC-003

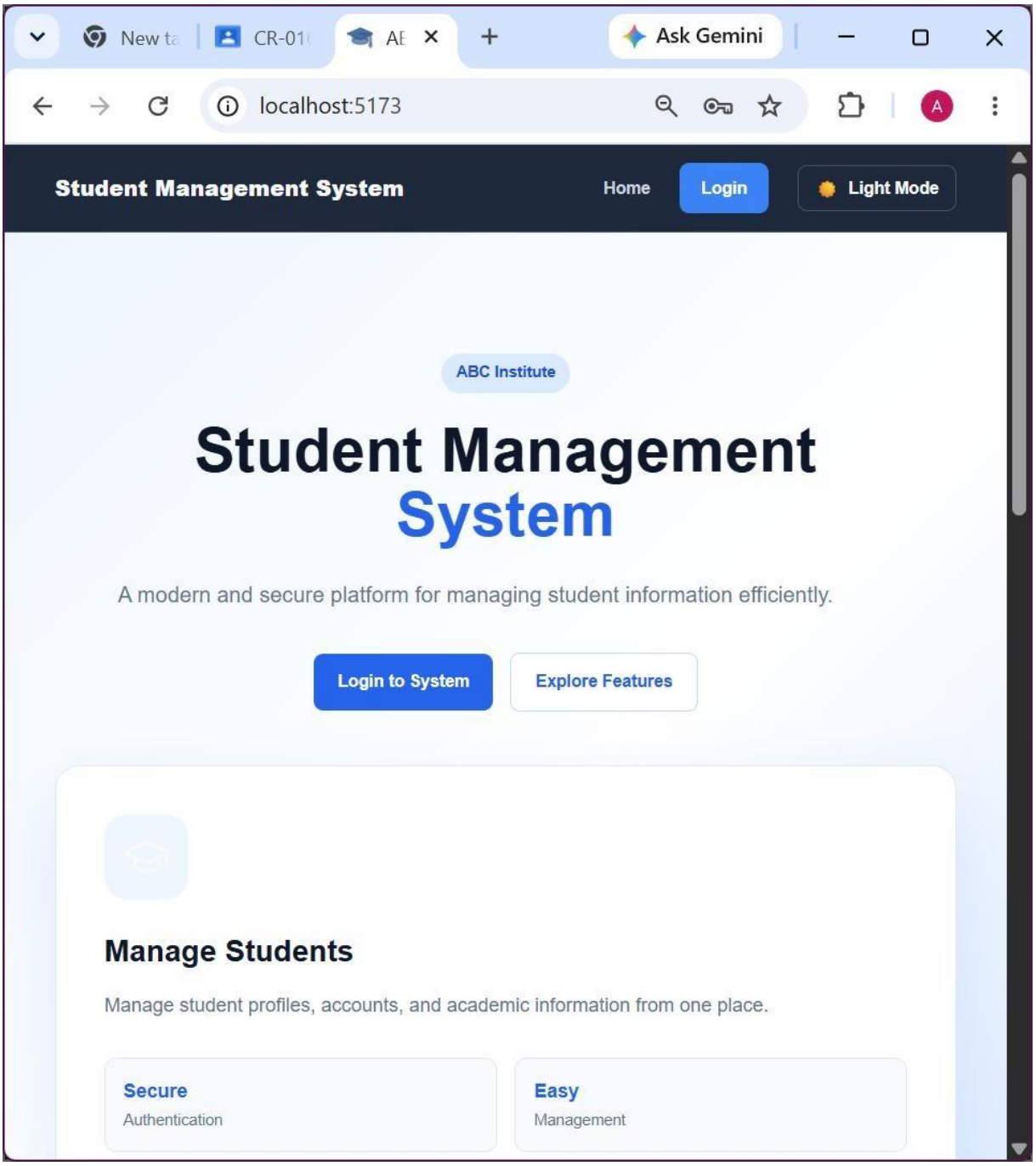


Figure 1 Student Management System landing page displaying the global Dark Mode theme

Evidence 02 – Administrator Dashboard in Dark Mode

Related Requirements : AC-005, AC-008.

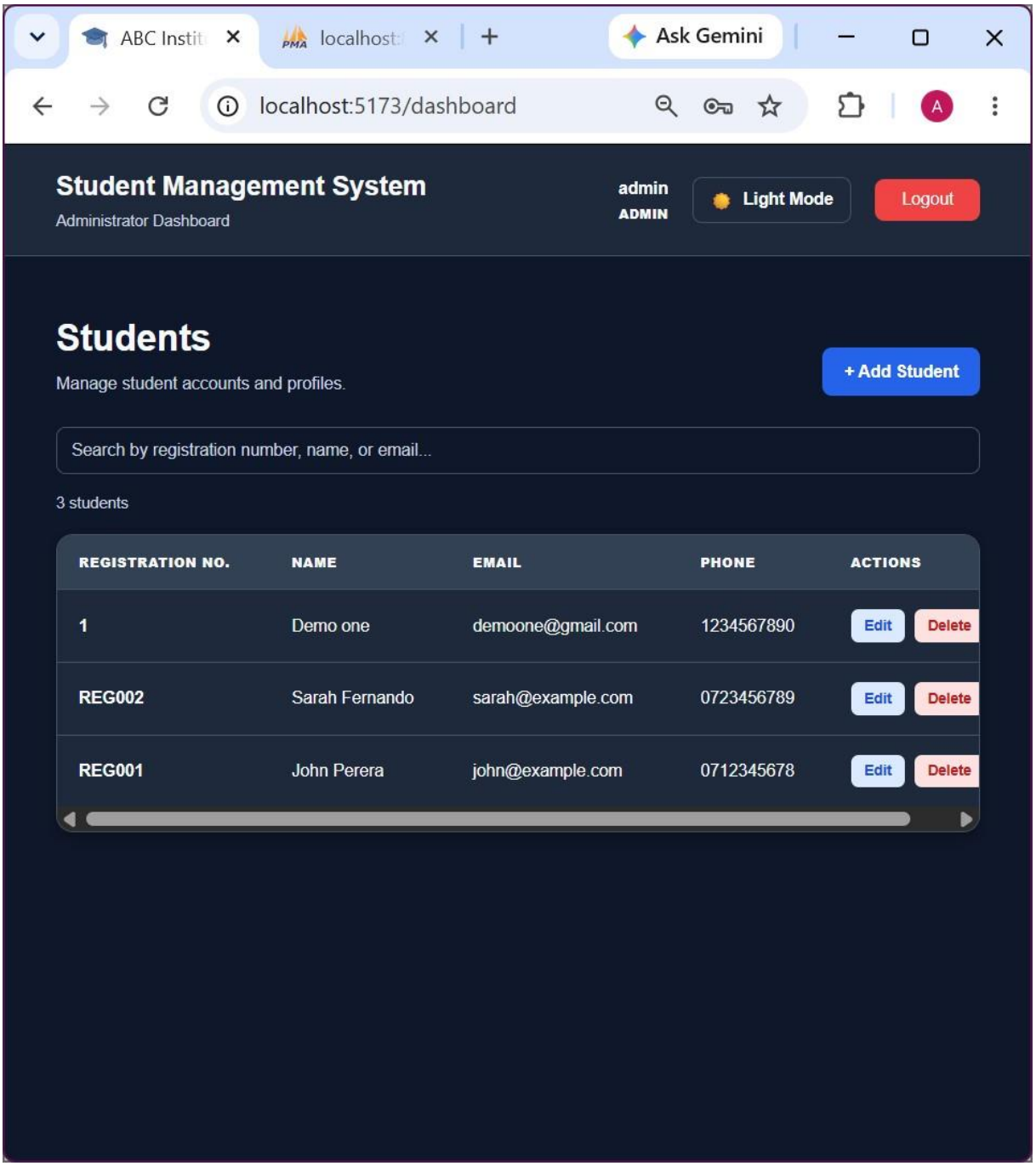


Figure 2 Administrator dashboard displaying Dark Mode with theme-aware student table styling.

6. Final Result

Item	Result
Requirements	Implemented; complete verification pending
Implementation	Completed
Testing	Partially Verified
Acceptance Criteria	Partially Verified
Existing CRUD	Regression verification pending
Theme Persistence	Implemented; runtime verification pending

Backend Changes	Not Required
Database Changes	Not Required
Git Commit	
Git Branch	JFICT09/CR012
Final Status	Pending Final Verification

6.1 Final Summary

CR-012 was implemented to introduce Light Mode and Dark Mode functionality throughout the Student Management System.

The implementation uses React Context API to provide centralized theme management and a reusable Theme Toggle component for switching between themes. CSS custom properties and theme-specific styling were used to apply the selected colour scheme across the landing page, login page, administrator dashboard, student dashboard, and Add/Edit Student modal.

Browser localStorage was used to persist the selected theme preference. Additional styling improvements were implemented to maintain appropriate contrast between backgrounds and text, including adjustments to the student table's registration number column.

The change was implemented at the frontend level without requiring modifications to the existing backend APIs or database structure. Existing authentication, student management, validation, and CRUD functionality were preserved in the implementation design.

Manual testing confirmed Dark Mode operation on the administrator dashboard and landing page, as well as the functioning of Add/Edit Student modals. Further verification is required for complete theme persistence, student dashboard behaviour, and regression testing before final acceptance.

The implementation was committed to the JFICT15/CR012 branch with commit ID cfe5f8d and is associated with the remote branch origin/JFICT15/CR012.

6.2 Issues / Limitations

During implementation, some pages and components initially retained fixed light-theme colours. These issues were addressed by adding theme-specific CSS styling.

A text-contrast issue was also identified in the administrator student table, where registration numbers appeared dark against the Dark Mode background. A targeted CSS correction was provided.

The following verification remains outstanding: confirmation of the registration number fix, full student dashboard theme testing, persistence testing after refresh, and regression testing of existing student operations.

No unresolved backend or database issues have been identified in the theme implementation.

No significant issues or limitations identified during testing.

Student Name: J.Thushany

Student ID: JF/ICT/24/09

Date: 9 October 2026

Signature: Thushany